ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

NT106 — LẬP TRÌNH MẠNG CĂN BẢN

CẨM NANG LABGUARD

HƯỚNG DẪN THÀNH VIÊN D

KIỂM THỬ TCP & ĐỊNH HƯỚNG THIẾT KẾ HỖ TRỢ XỬ LÝ SỰ CỐ

Đỗ Minh Ngọc: công cụ kiểm thử truyền thông TCP và chức năng hỗ trợ phân tích, xử lý sự cố tích hợp trong LabGuard

BIÊN SOẠN: NHÓM ĐỒ ÁN LABGUARD

TÀI LIỆU ĐỊNH HƯỚNG THIẾT KẾ KỸ THUẬT (RÀ SOÁT PR #14 — 18/10/2026)

TP. HỒ CHÍ MINH — 2026

KHỞI ĐỘNG NHANH DÀNH CHO THÀNH VIÊN D

(Bản định hướng thiết kế tinh gọn — nắm bắt nhiệm vụ và ranh giới kỹ thuật trong 2 phút)

1. BẠN LÀ AI?

Bạn là **Đỗ Minh Ngọc**, đảm nhiệm phân công chính thức: **Công cụ kiểm thử truyền thông TCP và chức năng hỗ trợ phân tích, xử lý sự cố tích hợp trong LabGuard** (mã nhóm nhiệm vụ: LG-D-001 đến LG-D-010). Vai trò của bạn là kỹ sư kiểm thử tầng mạng và người thiết kế thư viện phân tích bằng chứng sự cố nhúng trong Admin WinForms.

2. BẠN CẦN LÀM GÌ?

- **Kiểm thử TCP thiết yếu (Nền tảng):** Nắm chắc lập trình Socket không nghẽn, đóng khung 4 byte Big-Endian (`FrameCodec`), kiểm tra lưu lượng bằng Wireshark (port 5000); nghiệm thu TCP Echo (`LG-D-002` đã XONG qua PR #8) và hoàn thiện TCP Relay (`LG-D-003`) với hàng đợi Bounded Channel chống tràn RAM.
- **Thiết kế hỗ trợ xử lý sự cố (D-007..D-010):** Tự tay thiết kế và lập trình thư viện C# thuần túy (`LabGuard.Troubleshooting`) phục vụ Admin WinForms; thẩm định chất lượng dữ liệu (độ mới, phần thiếu/cũ, trạng thái kết nối do Central cung cấp); đối soát ma trận quy tắc tất định; phân định rõ 3 tầng trạng thái (Kết nối, Cảnh báo kỹ thuật, Quy trình xử lý); xếp hạng bước kiểm tra ưu tiên không xâm lấn; dứt khoát không đưa ra kết luận võ đoán khi thiếu bằng chứng.

3. BẠN TUYỆT ĐỐI KHÔNG CẦN LO PHẦN NÀO?

- Bạn **không cần** lo đo phần cứng máy trạm Windows hay vòng lặp Agent (việc của **Võ Trọng Phúc — A**).
- Bạn **không cần** tạo bảng SQL Server, không viết `RuleEngine` phát hiện sự cố máy chủ (việc của **Nguyễn Bá Quang — B**).
- Bạn **không cần** thiết kế giao diện WinForms Dashboard hay bắt sự kiện nút bấm UI (việc của **Lý Tuyết Vy — C**).
- *Phân định mở rộng:* Gateway L4, Collector L7 và Bộ giả lập Simulator 50 máy là **nghiên cứu mở rộng P1 tùy chọn** (nằm ở Phụ lục), không phải cam kết bắt buộc của bạn.

4. HÔM NAY BẮT ĐẦU TỪ ĐÂU?

Đọc kỹ quy chuẩn Wire Protocol (`docs/spec/PROTOCOL_SPEC.md`) và Quyết định Kiến trúc (`docs/spec/ARCHITECTURE_DECISIONS.md`); nắm vững các hợp đồng dữ liệu giữa Central và Admin; tự thiết kế cấu trúc lớp và thuật toán thẩm định bằng chứng cho thư viện phân tích; viết bộ unit test độc lập cho 6 kịch bản sự cố chuẩn hóa (Ca 1 đến Ca 6).

5. CÁC TÀI LIỆU VÀ TẬP TIN CẦN MỞ ĐẦU TIÊN:

1. `docs/internal/TEAM_ASSIGNMENTS.md`: Phân công chính thức và ranh giới tam giác B-C-D.
2. `docs/spec/PROTOCOL_SPEC.md`: Đặc tả 69 bản tin và định dạng WireEnvelope chuẩn.
3. `docs/spec/ARCHITECTURE_DECISIONS.md`: Các quyết định kiến trúc chính thức (CP2-001 đến CP2-019).
4. `src/LabGuard.Protocol/FrameCodec.cs`: Cách đóng gói và giải mã tiền tố 4 byte Big-Endian.
5. `docs/internal/WBS_TASKS.csv`: Tiêu chí nghiệm thu các mã LG-D-001..LG-D-010.

MỤC LỤC

Hiện trạng đối chiếu ngày 18/10/2026 **1**

1 Tổng quan vai trò và ranh giới kỹ thuật của Thành viên D **2**
1.1 Hiện trạng nhiệm vụ và lộ trình triển khai 2
1.2 Phân định ranh giới trách nhiệm tam giác (Quang B — Vy C — Ngọc D) 2
1.3 Bốn nguyên tắc bất biến của chức năng hỗ trợ xử lý sự cố 4

2 Nền tảng kiểm thử truyền thông TCP thiết yếu (D-001, D-002, D-003) **4**
2.1 Lập trình Socket TCP căn bản và công cụ TCP Echo (LG-D-002) 4
2.2 Đóng khung 4 byte và bảng byte minh họa 5
2.3 Đặc tả TCP Relay hai chiều và áp lực ngược (LG-D-003) 5

3 Trọng tâm D-007 đến D-010: Định hướng thiết kế hỗ trợ xử lý sự cố **6**
3.1 Mô hình đầu vào / đầu ra và căn cứ dữ liệu (LG-D-007) 6
3.2 Chu kỳ đo đạc, khoảng thời gian chuỗi mẫu và thẩm định chất lượng dữ liệu 9
3.3 Phân định rõ ràng 3 tầng trạng thái độc lập 10
3.4 Ma trận quy tắc đối soát và nguyên tắc xếp hạng bước kiểm tra (LG-D-008) 11
3.5 Thiết kế giao diện thư viện tham khảo và Tiêu chí nghiệm thu (LG-D-009) 12
3.6 Bộ sáu ca kiểm thử dự kiến (LG-D-010) 13

4 Năm bẫy lỗi lập trình cần tránh dành cho Thành viên D **14**

5 Bộ câu hỏi vấn đáp chuyên sâu dành cho Thành viên D (20 câu) **15**

Phụ lục: Nghiên cứu mở rộng P1 tùy chọn **18**

Hiện trạng đối chiếu ngày 18/10/2026

TÓM TẮT 30 GIÂY

Mốc đối chiếu: main `d4e20ad` sau PR #13; PR #14 đang Draft, nguồn đầu lượt sửa `64488a5`. PR #11 là mốc bổ sung Agent, không phải thay đổi mới nhất. Đây là cẩm nang học và kế hoạch hiện thực; các mô tả tương lai/pseudocode không phải bằng chứng tính năng đã hoạt động.

- **Agent đã có:** chương trình Console, cấu hình JSON, DeviceId riêng cho mỗi bản cài đặt, đo CPU/RAM/thời gian hoạt động Windows tại máy, chế độ thử TCP/HELLO, hủy tác vụ và ghi nhật ký.
- **Chế độ giám sát bị chặn có chủ đích** tới khi TLS/phiên kết nối/AUTH sẵn sàng. MetricsService/HeartbeatService đã có thành phần và kiểm thử nhưng chưa được bật trong chương trình khởi chạy.
- **Chưa hoàn tất:** A-005 kết nối lại, A-006 ReplayBuffer, đọc phản hồi/kiểm tra mất kết nối, tích hợp SQL/ACK và Windows Service. B-001 đã có bộ tiếp nhận TCP; B-005 mới có DDL, còn đang làm. C-001 có bố cục 0.1, chưa nghiệm thu liên kết dữ liệu và quy trình xử lý.
- **Kiểm thử đã báo cáo từ PR #11:** 195 bài đạt trên Windows tại máy: Protocol 107, Agent 64, Central 9, MockHarness 15. Đây **không phải kết quả GitHub CI**. Số 34/165 trong hồ sơ cũ là mốc lịch sử.

Thành viên	Trách nhiệm chính thức
A — Võ Trọng Phúc	Agent và Protocol.
B — Nguyễn Bá Quang	Central Server và SQL Server.
C — Lý Tuyết Vy	Admin Windows Forms.
D — Đỗ Minh Ngọc	Công cụ kiểm thử truyền thông TCP và chức năng hỗ trợ phân tích, xử lý sự cố tích hợp trong LabGuard.

Nguồn: quyết định trực tiếp của người điều phối dự án ngày 17/10/2026; không tự tạo biên bản họp hoặc chữ ký. Chức năng hỗ trợ xử lý sự cố là **phần việc chính thức trong kế hoạch**, chưa phải chức năng đã được lập trình. Logic nhúng trong quy trình xử lý sự cố của LabGuard Admin, dùng dữ liệu có thẩm quyền qua Central; D phối hợp B/C, không làm ứng dụng TCP độc lập.

Gateway/Collector và Simulator là **mở rộng P1 tùy chọn** sau khi phần cốt lõi ổn định. Nội dung mở rộng được giữ để học, không phải cam kết chính của Ngọc. GK là một buổi trình bày đề xuất theo lịch giảng viên, không phải hạn hoàn thành đồ án; các chặng là kế hoạch tương đối toàn dự án.

B-009 cần đối chiếu kế hoạch: WBS cũ nhắc AssignedToUserId/IncidentHistory nhưng schema hiện chỉ có AssigneeId và chưa có bảng lịch sử. Giữ bản ghi CHƯA LÀM để Quang/nhóm quyết định, không tự sửa schema hoặc coi bảng/cột cũ đã tồn tại.

Năm mốc khác nhau: kết nối TCP; xác thực ứng dụng; nhận số liệu; ghi thành công SQL; nhận ACK ứng dụng. Chế độ thử trả exit 0 chỉ cho biết lệnh kết thúc; phía bên kia đóng kết nối có thể chưa được phát hiện. Nhật ký kết nối không chứng minh đã xác thực hoặc lưu SQL.

Bắt đầu đúng cách: đọc `docs/agent_deployment/README.md` và `docs/internal/TEAM_ASSIGNMENTS.md`. WBS giữ 34 nhiệm vụ cũ và sáu nhiệm vụ bổ sung cho chức năng hỗ trợ xử lý sự cố, đều chưa làm. Nâng cấp .NET 10 là việc riêng toàn bộ dự án, chưa thực hiện trong lượt đồng bộ này.

GHI CHÚ QUAN TRỌNG

Windows-first: P0 hướng đến Agent thu số liệu bằng Win32, Admin Windows Forms và môi trường SQL Server trên Windows. Linux/macOS chỉ là mở rộng sau khi P0 ổn định; chưa yêu cầu viết collector đa nền tảng hoặc thay kiến trúc. Dòng kế hoạch cũ nhắc Ubuntu cần nhóm đối chiếu phạm vi riêng; lượt này không sửa WBS.

1. Tổng quan vai trò và ranh giới kỹ thuật của Thành viên D

TÓM TẮT 30 GIÂY

Thành viên D (**Đỗ Minh Ngọc**) phụ trách hai mảng công việc phối hợp: (1) **Công cụ kiểm thử truyền thông TCP** độc lập nhằm xác thực tầng mạng và điều tiết hàng đợi; (2) **Chức năng hỗ trợ phân tích, xử lý sự cố** được đóng gói thành thư viện C# dùng lại được tiêu thụ bởi LabGuard Admin. Sổ tay này đóng vai trò **định hướng thiết kế kỹ thuật**, đưa ra các yêu cầu, giao diện tham khảo và tiêu chí nghiệm thu để bạn tự tay thiết kế và lập trình hoàn thiện.

1.1 Hiện trạng nhiệm vụ và lộ trình triển khai

Căn cứ quyết định phân công chính thức ngày 17/10/2026 và bảng WBS hợp nhất:

- **LG-D-002 (TCP Echo Console):** Đã hoàn thành và nghiệm thu qua PR #8 (chương trình độc lập tại `tools/TcpEcho/Program.cs`).
- **LG-D-001 & LG-D-003 (Nền tảng TCP & TCP Relay Bounded Channel):** Chưa nghiệm thu (WBS: CHƯA LÀM); `tools/mock_harness` (PR #9) là công cụ bổ trợ kiểm thử, không thay thế mã nguồn chính thức của D-003.
- **LG-D-007 đến LG-D-010 (Hỗ trợ phân tích, xử lý sự cố tích hợp):** Là nhóm nhiệm vụ trọng tâm chính thức **đang ở giai đoạn thiết kế và chuẩn bị lập trình (CHƯA LÀM)**. Chức năng này bắt buộc phải là một thư viện C# thuần túy dùng lại được tiêu thụ bởi Admin WinForms, tuyệt đối không xây dựng thành ứng dụng chẩn đoán mạng chạy rời rạc.
- **LG-D-004, LG-D-005, LG-D-006 (Gateway, Collector, Simulator):** Là các hướng **nghiên cứu mở rộng P1 tùy chọn**, chỉ xem xét sau khi toàn bộ hệ thống cốt lõi đã nghiệm thu ổn định.

1.2 Phân định ranh giới trách nhiệm tam giác (Quang B — Vy C — Ngọc D)

Nhằm đảm bảo tính toàn vẹn kiến trúc và tránh chồng chéo trách nhiệm, ranh giới xử lý sự cố được quy định chặt chẽ theo Bảng 1:

Table 1: Phân định ranh giới trách nhiệm giữa Thành viên B, C và D

Thành viên	Phạm vi trách nhiệm có thẩm quyền	Điều tuyệt đối không làm
Nguyễn Bá Quang (Thành viên B)	- Lắng nghe kết nối TCP Agent, quản trị phiên kết nối. - Thực thi `RuleEngine` phát hiện sự cố dựa trên ngưỡng và hysteresis chính thức (CPU $\geq$ 90%, RAM $\geq$ 85%, nhịp tim quá hạn $>$ 20s). - Ghi nhận và lưu trữ CSDL SQL Server (`MetricSamples`, `Incidents`). - Cung cấp dữ liệu thiết bị và sự cố có thẩm quyền cho Admin.	- Không can thiệp logic giao diện hiển thị WinForms. - Không viết mã thuật toán gợi ý bước xử lý của Admin.
Đỗ Minh Ngọc (Thành viên D)	- Phát triển thư viện C# thuần túy (`LabGuard.Troubleshooting`). - Thẩm định chất lượng dữ liệu: độ mới (*freshness*), dữ liệu thiếu/cũ, trễ nhịp tim. - Đối chiếu ma trận quy tắc giải thích tất định (Rule-Based). - Xếp hạng danh mục các bước kiểm tra (ưu tiên không xâm lấn). - Dứt khoát từ chối kết luận suy đoán khi bằng chứng không đầy đủ.	- Không tự ý truy vấn trực tiếp vào CSDL SQL Server. - Không tự mở socket kết nối trực tiếp đến Agent. - Không tự ý đóng sự cố hoặc thay đổi dữ liệu Central. - Không nhúng mã giao diện WinForms vào thư viện.
Lý Tuyết Vy (Thành viên C)	- Thiết kế giao diện WinForms Admin Dashboard và bảng điều khiển sự cố. - Gọi thư viện của D để lấy nhận định và danh sách gợi ý kiểm tra. - Điều phối luồng thao tác của người quản trị (`INCIDENT_ACK`, `INCIDENT_START`, `INCIDENT_RESOLVE`). - Gửi lệnh cập nhật trạng thái sự cố về Central kèm kiểm soát khóa lạc quan OCC (`RowVersionHex`).	- Không tự ý viết mã phân tích quy tắc sự cố cứng trong WinForms Form. - Không tin cậy dữ liệu cũ khi người dùng thao tác.

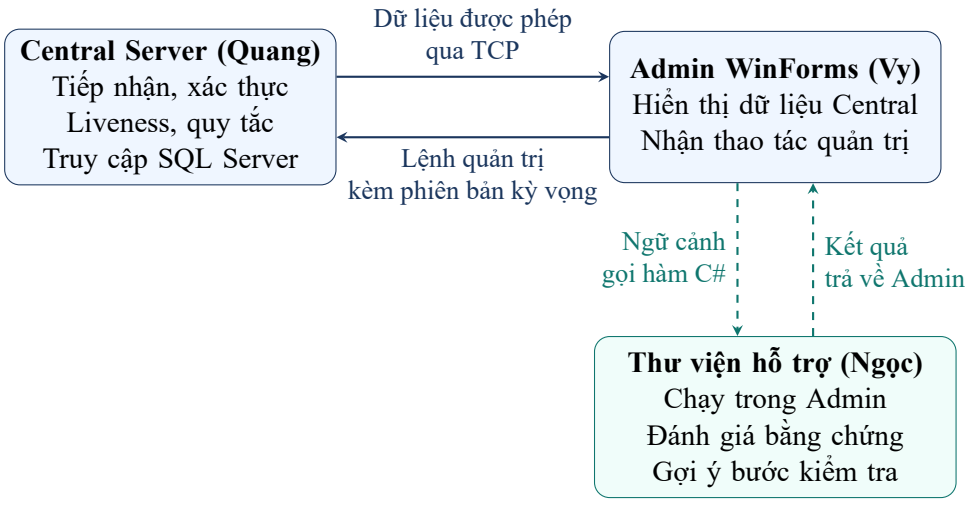


Figure 1: Sơ đồ phân định trách nhiệm tam giác B-C-D và luồng dữ liệu hỗ trợ phân tích sự cố

1.3 Bốn nguyên tắc bất biến của chức năng hỗ trợ xử lý sự cố

1. **100% Luật tất định (Deterministic Rule-Based), tuyệt đối cấm Machine Learning / Chatbot AI:** Hệ thống không sử dụng mạng nơ-ron, học máy hay gọi API mô hình ngôn ngữ lớn (LLM). Mọi gợi ý phải giải thích được từ quy tắc và nguồn bằng chứng đã nêu, đảm bảo tính ổn định và khả năng tái lập trong kiểm thử.
2. **Tôn trọng hợp đồng dữ liệu thực tế:** Chỉ phân tích các trường dữ liệu thực sự có trong giao thức mạng (`WireEnvelope`) và CSDL SQL Server (`MetricSamples`, `Incidents`). Các thông tin chưa hỗ trợ (như danh sách tiến trình con `ProcessList` trên máy trạm) bắt buộc phải ghi rõ là *Chưa có nguồn dữ liệu / Dự kiến*, tuyệt đối không bịa đặt dữ liệu mẫu.
3. **Tuyệt đối không bịa đặt điểm tin cậy ảo:** Không xuất các chỉ số phần trăm vô căn cứ (ví dụ: "khả năng lỗi CPU là 87%"). Kết luận phải là mệnh đề logic rõ ràng: *Đủ bằng chứng xác nhận*, *Nghi vấn cần kiểm chứng*, hoặc *Không đủ bằng chứng*.
4. **Dứt khoát từ chối kết luận khi thiếu dữ liệu:** Khi nguồn metric thiếu, cũ hoặc chưa đủ mẫu liên tiếp, thư viện nêu giới hạn bằng chứng và không kết luận tải hiện tại. Nếu Central báo OFFLINE do heartbeat hợp lệ quá hạn, hiển thị riêng tình trạng mất liên lạc; không tự lấy mốc 20 giây làm ngưỡng cũ của metric.

2. Nền tảng kiểm thử truyền thông TCP thiết yếu (D-001, D-002, D-003)

TÓM TẮT 30 GIÂY

Trước khi tích hợp logic xử lý sự cố, Thành viên D phải làm chủ tầng truyền vận mạng: nắm chắc cơ chế bắt tay 3 bước TCP, lập trình socket không nghẽn, kiểm soát hiện tượng phân mảnh luồng byte bằng tiền tố độ dài 4 byte Big-Endian, và áp dụng hàng đợi Bounded Channel để điều tiết áp lực ngược (Backpressure).

2.1 Lập trình Socket TCP căn bản và công cụ TCP Echo (LG-D-002)

TCP là giao thức hướng luồng byte (*Byte Stream*), không có ranh giới bản tin tự nhiên. Khi bên gửi gọi lệnh `Write`, dữ liệu có thể bị phân mảnh (*packet fragmentation*) hoặc dồn ép (*packet coalescing*) tùy thuộc vào kích thước MSS và thuật toán Nagle của hệ điều hành.

Chương trình `TcpEcho` (PR #8) chứng minh khả năng thiết lập máy chủ phản hồi tại `tools/TcpEcho/Program.cs`:

- Lắng nghe trên cổng cục bộ bằng `new TcpListener(IPAddress.Any, port)`.
- Tiếp nhận kết nối bất đồng bộ qua `await listener.AcceptTcpClientAsync(ct)`.
- Đọc luồng dữ liệu từ `NetworkStream` vào bộ đệm và ghi trả nguyên vẹn về máy gửi.
- **Giải phóng tài nguyên chuẩn mực:** Đóng `TcpClient` bằng `using` và giải phóng `NetworkStream`, bắt chính xác các ngoại lệ `IOException` và `OperationCanceledException` khi đối tác ngắt kết nối đột ngột.

2.2 Đóng khung 4 byte và bảng byte minh họa

`FrameCodec` dùng tiền tố **UInt32 Big-Endian**: $0 < N \leq 262\,144$ byte UTF-8, rồi đúng N byte payload. Không tính 4 byte tiền tố vào N.

Ví dụ tự dựng, không phải gói bắt thực tế: chuỗi JSON dưới đây là một dòng ASCII/UTF-8, không BOM, không khoảng trắng hoặc xuống dòng bổ sung. Việc tự ngắt dòng khi in không thuộc dữ liệu wire.

```
{"version":1,"messageType":"HEARTBEAT","requestId":"550e8400-e29b-41d4-a716
    -446655440000","deviceId":"MAY-03","sentAtUtc":"2026-10-18T08:30:00Z","payload
    ":{"sequenceNumber":1,"uptimeSeconds":1234}}
```

Listing 1: HEARTBEAT tự dựng dùng để tính byte

Table 2: Offset minh họa tính từ byte đầu khung đã ghép lại, không phải đầu gói Ethernet

Offset bao gồm hai đầu	Số byte	Nội dung
`0x0000--0x0003`	4	`00 00 00 C5`: $N = 197$; đọc bằng `ReadUInt32BigEndian`.
`0x0004--0x0010`	13	`version`
`0x0011--0x002A`	26	`messageType`
`0x002B--0x005D`	51	`requestId`
`0x005E--0x0071`	20	`deviceId`
`0x0072--0x0094`	35	`sentAtUtc`
`0x0095--0x00C8`	52	`payload`

Tổng: $4 + 197 = 201$ byte. Các đoạn liên tiếp phủ đúng offset 0 đến 200, không trùng hoặc bỏ byte. SHA-256 toàn khung (nối hai dòng, không thêm ký tự):

```
7b318078cb7239743e64bf212b1858bf
bf17534e3f066849fd151a9d74f00ef9
```

Khi kiểm thử sau này, lọc `tcp.port == 5000` và đối chiếu luồng TCP đã tái ghép; một khung có thể nằm trong nhiều segment. Với TLS, payload bắt trên mạng là dữ liệu mã hóa: chỉ xem JSON ở endpoint phát triển TCP thường hoặc qua giải mã được phép. Bảng này không chứng minh đã có capture hay đã kiểm thử LAN.

2.3 Đặc tả TCP Relay hai chiều và áp lực ngược (LG-D-003)

Đây là yêu cầu để Ngọc tự thiết kế, không phải mã Relay đã hoàn tất. Đối chiếu trực tiếp `src/LabGuard.Protocol/FrameCodec.cs`:

```
Task<byte[]?> ReadFrameAsync(Stream stream,
    int maxPayloadSize = MaxPayloadSize,
    CancellationToken ct = default)
Task WriteFrameAsync(Stream stream,
    ReadOnlyMemory<byte> payload,
    CancellationToken ct = default)
```

Listing 2: Chữ ký hiện có, không phải thuật toán Relay

`ReadFrameAsync(source, ct: ct)` là cách truyền token theo tên; tham số thứ hai theo vị trí là `int`, không phải token. Kết quả chỉ là **payload**, đã bỏ tiền tố 4 byte. `null` là EOF sạch tại ranh giới khung; EOF giữa header/payload gây `EndOfStreamException`; độ dài 0 hoặc quá giới hạn gây `InvalidDataException`.

- Mỗi chiều có một hàng đợi riêng: client–đích và đích–client. Theo CP2-019, mỗi hàng đợi có tối đa 100 phần tử, `FullMode.Wait`; không dùng chung hàng đợi giữa hai chiều hoặc hai phiên.
- Hàng đợi chứa payload nguyên byte. Writer gọi `WriteFrameAsync(destination, payload, ct)` để tạo lại đúng *một* tiền tố; không ghi payload trần và không đóng khung hai lần.
- Chỉ một writer ghi mỗi stream. Khi đầy, producer chờ bất đồng bộ; không bỏ khung để giảm tải. Giới hạn số phần tử không phải giới hạn chính xác toàn bộ RAM vì còn buffer và đối tượng đang xử lý.
- Hai chiều chạy đồng thời để phản hồi không bị chặn bởi chiều yêu cầu. Ngọc phải định nghĩa EOF/half-close, hoàn tất hàng đợi, xả dữ liệu đã nhận và hủy khi lỗi; không để tác vụ chờ vô hạn.

```
CHO MOI CHIEU (client -> dich, dich -> client), CHAY DONG THOI:
  reader: doc mot payload hop le; cho cho trong hang doi
  writer: lay payload; ghi lai frame voi dung mot length prefix
  EOF sach: hoan tat hang doi, xa du lieu theo chinh sach half-close
  loi/huy: bao loi, huy cac tac vu lien quan, giai phong ket noi
```

Listing 3: Pseudocode định hướng, không cung cấp cài đặt hoàn chỉnh

Tiêu chí kiểm thử dự kiến: tách header/payload qua nhiều lần đọc; ghép nhiều khung; so nguyên byte trước/sau cả hai chiều; giữ phản hồi khi phía kia còn gửi; queue đầy phải chờ; hủy khi chờ không treo; EOF sạch/EOF giữa khung; độ dài 0/quá lớn; writer không xen byte; các phiên độc lập. Chính sách half-close và giới hạn chờ đóng cần được ghi rõ trước nghiệm thu. Chưa có kết quả Relay mới trong tài liệu này.

3. Trọng tâm D-007 đến D-010: Định hướng thiết kế hỗ trợ xử lý sự cố

TÓM TẮT 30 GIÂY

Đây là nội dung kỹ thuật cốt lõi của Thành viên D. Mục tiêu phần này là cung cấp **định hướng thiết kế, hợp đồng dữ liệu tham khảo, thuật toán phân tích và tiêu chí nghiệm thu** để bạn Đỗ Minh Ngọc tự tay triển khai thư viện `LabGuard.Troubleshooting`. Thư viện tiếp nhận ngữ cảnh sự cố từ Admin, thẩm định độ mới dữ liệu, áp dụng ma trận quy tắc tất định, phân định rạch ròi 3 tầng trạng thái và đề xuất các bước kiểm tra có thứ tự ưu tiên logic.

3.1 Mô hình đầu vào / đầu ra và căn cứ dữ liệu (LG-D-007)

Thư viện được thiết kế theo nguyên lý Clean Architecture: là một thư viện lớp C# thuần túy (*Class Library*), không phụ thuộc vào Windows Forms UI, không kết nối trực tiếp CSDL và không mở socket mạng. Toàn bộ dữ liệu trao đổi được chuẩn hóa qua các đối tượng DTO.

3.1.1 Cấu trúc DTO Đầu vào tham khảo: `TroubleshootingContext`

Dữ liệu đầu vào được Admin tổng hợp từ các bản tin có thẩm quyền của Central và chuyển giao cho thư viện:

- **DeviceState (Ánh xạ từ `DeviceSummaryDto` trong `src/LabGuard.Protocol/Dtos/AdminDtos.cs`):**
  - `DeviceId` (string, bắt buộc): Định danh máy trạm (ví dụ: `"AGENT-LAB01-03"`).
  - `Hostname` (string): Tên Windows máy trạm (lưu ý: tài liệu gốc dùng `Hostname`, không phải `MachineName`).
  - `IpAddress` / `MacAddress` (string): Địa chỉ IP kết nối và địa chỉ MAC.
  - `Status` (string, bắt buộc): Trạng thái thiết bị trên Central gồm `"ONLINE"`, `"OFFLINE"`, hoặc `"DEGRADED"` (không dùng cờ boolean `IsConnected`).
  - `LastSeenUtc` (string? trên wire): thông tin quan sát do Central cung cấp. DTO chỉ khai báo trường nullable, không chứng minh đó là thời điểm heartbeat hợp lệ cuối; không tự chuyển thành đồng hồ liveness trong Admin.
  - `CpuUsage` / `RamUsage` (double?): Chỉ số tải gần nhất lưu trên cache máy chủ.
  - `UptimeSeconds` (long?): Thời gian máy trạm đã hoạt động liên tục (tính bằng giây).
  - *Trường mở rộng tùy chọn:* `AgentVersion` (string?) được xem là thuộc tính nội bộ mở rộng, không nằm trong wire contract P0.
- **IncidentRecord (Ánh xạ từ `IncidentSummaryDto` trong `src/LabGuard.Protocol/Dtos/AdminDtos.cs`):**
  - `IncidentId` (long, bắt buộc): Mã sự cố định danh duy nhất.
  - `DeviceId` (string, bắt buộc): Mã máy trạm xảy ra sự cố.
  - `RuleName` (string, bắt buộc): Tên quy tắc sự cố chính thức theo ADR CP2-006..CP2-008: `"HIGH_CPU"`, `"HIGH_MEMORY"`, `"SERVICE_DOWN"`, hoặc `"DEVICE_OFFLINE"` (lưu ý: giao thức dùng `RuleName`, không dùng `IncidentType`).
  - `ConditionState` (string): Giá trị thiết kế theo ADR (DTO hiện chú thích ACTIVE/RECOVERED; UNKNOWN cần B/C xác minh ánh xạ trước tích hợp): `"ACTIVE"`, `"RECOVERED"`, hoặc `"UNKNOWN"`.
  - `WorkflowState` (string): Nhãn thiết kế quy trình; DTO/đặc tả đang khác tên như ghi chú đối chiếu bên dưới: `"OPEN"`, `"ACKNOWLEDGED"`, `"INVESTIGATING"` (hoặc `"IN_PROGRESS"`), `"RESOLVED"`.
  - `OccurrenceCount` (int): Số lần điều kiện cảnh báo lặp lại.
  - `OpenedAtUtc` (string, bắt buộc): Thời điểm sự cố được ghi nhận lần đầu.
  - `RowVersionHex` (string, bắt buộc): Chuỗi hex 18 ký tự đại diện cho token phiên bản SQL Server OCC (ví dụ: `"0x0000000000000001"`).
- **RecentMetrics (Danh sách các mẫu đo gần nhất):**
  - Danh sách nội bộ dự kiến, không phải trường sẵn có của `DeviceSummaryDto`. Mẫu có thể tham khảo `MetricSampleDto` (`TelemetryDtos.cs`), gồm `MetricName` (`"CPU"`, `"RAM"`), `MetricValue` (double), `Unit` (`"%"`), và `RecordedAtUtc` (string ISO 8601 UTC).
- **SystemTimeUtc (DateTime):** giờ UTC dùng đánh giá hiển thị; không phải đồng hồ đơn điệu của Central.

3.1.2 Nguồn ba mẫu: P0 không đồng nghĩa có API lịch sử

P0 bắt buộc có `LIST_DEVICES/DEVICE_PAGE`, snapshot và `DEVICE_UPDATE`. `DeviceSummaryDto` có số CPU/RAM gần nhất và `LastSeenUtc`; `DeviceUpdatePayload` có `UpdatedAtUtc`. Không

DTO nào trong hai loại này chứa lịch sử ba mẫu hoặc `RecordedAtUtc` của mẫu gốc. Ba lần poll/snapshot, ba lần nhận event hay giá trị lặp lại không chứng minh ba mẫu liên tiếp.

1. **Giai đoạn 1 – chỉ dùng dữ liệu P0 được Central cho phép:** khi các handler cốt lõi sẵn sàng, Admin hiển thị giá trị hiện có và sự cố Central đã ghi. Nếu không có chuỗi mẫu thì để `RecentMetrics` rỗng/không sẵn có, ghi `Insufficient` cho câu hỏi xu hướng tải; chỉ giải thích "Central báo HIGH_CPU", không tự xác nhận lại ba mẫu. Bản ghi cục bộ thời điểm Admin nhận event chỉ là lịch sử quan sát, không được đổi tên thành thời điểm đo hay lịch sử SQL. Hiện listener HELLO chưa chứng minh các handler này đã hoàn tất.
2. **Giai đoạn 2 – có điều kiện nhóm chấp thuận:** `GET_METRICS/METRICS_RESULT` thuộc *OPTIONAL*, không phải P0 bắt buộc. Tên bản tin không chứng minh đã có handler/API lịch sử. Quang cần xác minh nguồn SQL, quyền, hợp đồng phản hồi; Vy ánh xạ vào ngữ cảnh nội bộ; Ngọc chỉ dùng dữ liệu đã xác minh. Không tự thêm trường hoặc sửa wire/schema đóng băng.
3. **Điều kiện gọi là ba mẫu liên tiếp:** đúng thiết bị, cùng nguồn/phiên phù hợp, đúng loại chỉ số, có thứ tự và thời điểm đo xác minh được, không lặp do snapshot/replay và không có khoảng thiếu chưa giải thích. Nếu phản hồi chưa đủ thông tin để chứng minh tính liên tiếp thì vẫn báo thiếu bằng chứng, không tự tạo timestamp/BootId.

Phụ thuộc: LG-B-010 đối chiếu dữ liệu/nguồn sau khi phần Central cốt lõi sẵn sàng; LG-C-012 ghép ngữ cảnh và giao diện sau khi phần Admin cốt lõi sẵn sàng. Ước lượng 2 giờ/3 giờ là phần ghép bổ sung, không bao gồm xây API lịch sử hoặc hoàn thiện lõi còn thiếu. Unit test được dùng fixture tổng hợp ba mẫu, phải ghi rõ không phải lịch sử lấy từ Central thật.

GHI CHÚ QUAN TRỌNG

GHI CHÚ ĐỐI SOÁT HỢP ĐỒNG & ĐIỂM CẦN THỐNG NHẤT TRONG NHÓM:

1. **Tên trạng thái quy trình xử lý:** Trong DTO `AdminDtos.cs` sử dụng tên `"INVESTIGATING"`, trong khi ADR CP2-012 và `PROTOCOL_SPEC.md` liệt kê `"IN_PROGRESS"`. Đây là chênh lệch cần B/C thống nhất adapter trước tích hợp; Ngọc không tự nhận cả hai là wire enum hợp lệ hoặc tự sửa giao thức đóng băng.
2. **Định danh người phụ trách sự cố:** Trong `AdminDtos.cs` là thuộc tính `AssignedToUserId` kiểu `int?`, trong khi lược đồ SQL DDL hiện hành đặt tên cột là `AssigneeId` kiểu `NVARCHAR(64)`. Sổ tay ghi nhận đây là vấn đề mở giữa Quang (B) và Vy (C); Ngọc chỉ nhận giá trị hiển thị mà không phụ thuộc cứng vào kiểu dữ liệu này.
3. **Trường dữ liệu tiến trình (`ProcessList`):** Giao thức P0 hiện tại **hoàn toàn chưa có** bản tin thu thập danh sách tiến trình từ Agent. Do đó, mọi gợi ý liên quan đến tiến trình bắt buộc phải hướng dẫn người quản trị kiểm tra thủ công tại máy trạm, tuyệt đối không giả định phần mềm đã có dữ liệu này.

3.1.3 Cấu trúc DTO Đầu ra tham khảo: `TroubleshootingAssessment`

- `DataQuality` là enum *nội bộ đề xuất*, chưa nằm trong DTO giao thức: `Fresh`, `Stale`, `Insufficient`, `Missing`. Tách khỏi kết nối và trạng thái chuyển tiếp; không thêm enum này vào wire.
- `EvidenceSummary` (string): Tóm lược định lượng các dữ kiện khách quan ghi nhận được (số mẫu, giá trị đo, thời gian trễ).
- `AssessmentConclusion` (string): Nhận định kỹ thuật có giới hạn, nêu rõ phạm vi và từ chối suy đoán khi thiếu căn cứ.

- `RecommendedChecks` (`IReadOnlyList<RecommendedCheck>`): Danh mục các bước kiểm tra thực tế đã được xếp hạng ưu tiên (gồm `StepNumber`, `Title`, `Description`, `Priority`, `IsNonInvasive`).
- `PermittedActions` (`IReadOnlyList<string>`): Gợi ý các lệnh quản trị phù hợp với trạng thái sự cố và quyền RBAC của người dùng (`INCIDENT_ACK`, `INCIDENT_START`, `INCIDENT_RESOLVE`).

3.2 Chu kỳ đo đạc, khoảng thời gian chuỗi mẫu và thẩm định chất lượng dữ liệu

3.2.1 Thời điểm đo, quan sát và liveness có ý nghĩa khác nhau

- `RecordedAtUtc`: thời điểm lấy mẫu do Agent ghi; `sentAtUtc`: thời điểm gửi envelope. Cả hai phụ thuộc đồng hồ client; mẫu replay không chứng minh Agent đang sống.
- `LastSeenUtc`: trường UTC nullable trong `DeviceSummaryDto`; ý nghĩa cập nhật phải được B xác nhận. `UpdatedAtUtc` của event chỉ là thời điểm cập nhật event, không được giả làm thời điểm đo hoặc heartbeat cuối.
- Heartbeat hợp lệ: thuộc phiên Agent đã xác thực, đúng thế hệ kết nối và qua kiểm tra Central. HELLO đơn lẻ hoặc gói từ phiên cũ không đủ căn cứ phục hồi kết nối.
- Liveness Central dùng đồng hồ đơn điệu (CP2-ADR-03). Mốc cuối là heartbeat hợp lệ trong phiên hiện hành: $\Delta t = \text{elapsed}(t_{\text{now}}, t_{\text{lastHB}})$. Quá hạn khi $\Delta t > 20$ giây; quét dự kiến mỗi 2,5 giây. Admin dùng trạng thái do Central cung cấp, không tự suy ra OFFLINE từ phép trừ giờ UTC hoặc timestamp client. Mốc đơn điệu này chưa có trong DTO Admin.

Chu kỳ lấy mẫu dự kiến 5 giây; ba mẫu liên tiếp cách đều có khoảng đầu–cuối khoảng 10 giây. Đây không phải độ mới của mẫu: chuỗi 10 giây có thể đã cũ nhiều phút. Khi đồng hồ lệch hoặc nguồn/phiên không xác minh được, ghi lý do thiếu căn cứ; không lấy giờ nhận thay cho giờ đo. Ngưỡng cũ metric và dung sai clock skew cần nhóm thống nhất riêng, không tự dùng 20 giây của liveness làm hạn metric.

3.2.2 Hợp đồng chất lượng dữ liệu nội bộ đề xuất

Hai sổ tay dùng cùng bốn giá trị dưới đây; Ngọc tự thiết kế thuật toán, không có enum triển khai mới trong lượt sửa này.

`Missing` Thiếu nguồn hoặc định danh bắt buộc cho câu hỏi đang xét. Không có sự cố mở có thể là tình huống bình thường, không tự là Missing.

`Stale` Bằng chứng cần cho nhận định hiện tại đã cũ theo chính sách độ mới được nhóm chốt. OFFLINE là trạng thái kết nối riêng, không phải tên DataQuality.

`Insufficient` Có dữ liệu nhưng chưa đủ số mẫu/tính liên tiếp hoặc nguồn/thời điểm để kết luận; P0 chỉ có giá trị gần nhất thường thuộc trường hợp này khi xét xu hướng tải.

`Fresh` Đủ bằng chứng liên quan, có nguồn và độ mới được xác minh. Với kết luận chuỗi CPU/RAM phải đủ ba mẫu liên tiếp, không chỉ ba event.

Nếu một đánh giá cần một enum duy nhất, đề xuất ưu tiên `Missing`, rồi `Stale`, rồi `Insufficient`, cuối cùng `Fresh`; nêu đầy đủ các lý do đồng thời trong `EvidenceSummary`. Dữ liệu mâu thuẫn là

lý do cần kiểm tra, không tự thêm giá trị `Conflicting`. Quy tắc ưu tiên và chính sách độ mới là thiết kế nội bộ chờ nhóm thống nhất, không phải thay đổi wire.

Tiêu chí dự kiến: dữ liệu thiếu/cũ không cho phép kết luận tải hiện tại; vẫn có thể thuật lại sự cố Central đã báo và đề nghị bước kiểm tra phù hợp. Trạng thái chuyển tiếp trong Relay (đang nối/chờ/đã đóng/lỗi) chỉ là chẩn đoán công cụ nội bộ, không là DataQuality hay kết nối Agent do Central quyết định.

3.3 Phân định rõ ràng 3 tầng trạng thái độc lập

Hệ thống LabGuard được thiết kế trên mô hình máy trạng thái phân tầng độc lập (Dual-Layer FSM kết hợp Transport Liveness theo CP2-ADR-05). Thành viên D cần nắm vững bản chất từng tầng như mô tả tại Bảng 3: Các nhãn UNKNOWN/IN_PROGRESS theo thiết kế ADR không chứng minh handler/adapter đã hỗ trợ; khác biệt với chú thích DTO phải được B/C chốt. Trạng thái Relay không thuộc ba trục nghiệp vụ này.

Table 3: So sánh bản chất và chu kỳ vòng đời của 3 tầng trạng thái trong LabGuard

Tầng trạng thái	Khái niệm kỹ thuật	Các giá trị trạng thái	Cơ chế kích hoạt & chuyển đổi
Tầng 1: Kết nối thiết bị (Device Status)	Tình trạng kênh truyền socket và nhịp tim giữa Agent và Central.	- `ONLINE` - `OFFLINE` - `DEGRADED`	Central đo bằng đồng hồ đơn điệu: nếu không nhận `HEARTBEAT` quá hạn $>$ 20.0s (4 chu kỳ 5s) thì chuyển sang `OFFLINE`.
Tầng 2: Điều kiện kỹ thuật (ConditionState)	Tình trạng vượt ngưỡng tài nguyên tại máy trạm, do Central tính toán tự động.	- `ACTIVE` - `RECOVERED` - `UNKNOWN`	Tính toán theo thuật toán Hysteresis: - CPU $\geq$ 90% (3 mẫu) $\rightarrow$ `ACTIVE`. - CPU $\leq$ 75% (3 mẫu) $\rightarrow$ `RECOVERED`. - Mất mạng $\rightarrow$ `UNKNOWN`.
Tầng 3: Quy trình vận hành (WorkflowState)	Vòng đời sự cố nghiệp vụ lưu trữ bền vững trong bảng `Incidents`.	- `OPEN` - `ACKNOWLEDGED` - `INVESTIGATING` - `RESOLVED`	Do người quản trị thao tác qua Admin WinForms bằng các lệnh `INCIDENT_ACK`, `INCIDENT_START`, `INCIDENT_RESOLVE` kèm kiểm soát OCC.

GHI CHÚ QUAN TRỌNG

LƯU Ý CỐT TỬ VỀ ĐIỀU KIỆN PHỤC HỒI (`RECOVERED`): Khi tài nguyên máy trạm hạ nhiệt an toàn (CPU $\leq$ 75% trong 3 mẫu liên tiếp), Tầng 2 tự động chuyển điều kiện sang `RECOVERED`. Tuy nhiên, sự kiện này **TUYỆT ĐỐI KHÔNG TỰ ĐỘNG ĐÓNG SỰ CỐ THÀNH RESOLVED** ở Tầng 3. Sự cố vẫn giữ nguyên trạng thái trên bảng điều khiển Admin cho đến khi người quản trị xem xét nguyên nhân và chủ động nhấn nút giải quyết sự cố.

3.4 Ma trận quy tắc đối soát và nguyên tắc xếp hạng bước kiểm tra (LG-D-008)

Thư viện áp dụng ma trận đối soát quy tắc logic nhị phân với các nhận định kỹ thuật có giới hạn được mô tả tại Bảng 4:

Table 4: Ma trận quy tắc dự kiến; chuỗi mẫu chỉ dùng khi đã xác minh nguồn theo Mục 3.1.2

Tình huống	Bằng chứng cần có	Giới hạn nhận định	Bước kiểm tra gợi ý
CPU cao kéo dài (`HIGH_CPU`)	- 3 mẫu liên tiếp: CPU $\geq$ 90%. - Trạng thái: `ONLINE`. - Nguồn và độ mới mẫu đã xác minh; ONLINE do Central báo.	Tải tính toán cao liên tục trong cửa sổ quan trắc. Phù hợp điều kiện `ACTIVE` của quy tắc `HIGH_CPU`. Chưa có dữ liệu tiến trình để chỉ định nguyên nhân cụ thể.	1. (*Không xâm lấn*) Trao đổi với sinh viên về tác vụ thực hành đang chạy. 2. (*Quan sát*) Kiểm tra Task Manager trực tiếp tại máy trạm. 3. (*Can thiệp*) Yêu cầu sinh viên dừng tiến trình nếu không thuộc nội dung bài học.
Đột biến CPU (Transient Spike)	- 1 mẫu đo tăng cao $\geq$ 90%. - Các mẫu tiếp theo tự hạ $\leq$ 75%. - Trạng thái: `ONLINE`.	Đột biến tải nhất thời trong thời gian ngắn rồi tự hồi phục về ngưỡng an toàn. Không thỏa mãn điều kiện chuỗi mẫu để mở sự cố.	1. (*Không xâm lấn*) Tiếp tục theo dõi chu kỳ thu thập kế tiếp. 2. (*Ghi nhận*) Không cần thao tác can thiệp kỹ thuật.
Bộ nhớ RAM cao (`HIGH_MEMORY`)	- 3 mẫu liên tiếp: RAM $\geq$ 85%. - CPU: Ổn định bình thường. - Trạng thái: `ONLINE`.	Bộ nhớ vật lý sử dụng vượt ngưỡng an toàn kéo dài. Chưa đủ căn cứ lịch sử để khẳng định có rò rỉ bộ nhớ hay do ứng dụng thực hành chiếm dụng nhiều RAM.	1. (*Không xâm lấn*) Đối chiếu thời gian hoạt động (`UptimeSeconds`). 2. (*Quan sát*) Mở Task Manager tại máy kiểm tra ứng dụng chiếm RAM lớn. 3. (*Can thiệp*) Hướng dẫn sinh viên đóng bớt ứng dụng không cần thiết.
Mất kết nối đột ngột (`DEVICE_OFFLINE`)	- Trạng thái: `OFFLINE`. - Central báo heartbeat hợp lệ quá hạn $>$ 20 giây. - Mẫu đo trước đó: Bình thường.	Máy chủ mất tín hiệu liên lạc từ máy trạm. Điều kiện chuyển sang `UNKNOWN`. Không tự kết luận đứt cáp hay hỏng switch.	1. (*Không xâm lấn*) Người trực dùng công cụ ping được phép; không giả định Admin đã có lệnh này. 2. (*Vật lý*) Kiểm tra đèn tín hiệu cổng mạng LAN tại máy trạm. 3. (*Nguồn*) Kiểm tra người dùng có tắt máy hoặc khởi động lại máy không.
Tải cao trước khi ngắt kết nối	- Các mẫu đo cuối: CPU $\geq$ 90%, RAM $\geq$ 85%. - Sau đó: Central báo `OFFLINE` do heartbeat hợp lệ quá hạn.	Ghi nhận tình trạng tài nguyên tăng cao ngay trước khi mất liên lạc mạng. Chưa đủ bằng chứng xác định quan hệ nhân quả; cần kiểm tra tại chỗ.	1. (*Quan sát*) Quan sát màn hình máy trạm tại chỗ xem máy có bị đơ cứng chuột hoặc màn hình xanh không. 2. (*Can thiệp*) Nếu máy mất phản hồi hoàn toàn, kỹ thuật viên xác nhận và khởi động lại máy tại chỗ.

Tình huống	Bằng chứng cần có	Giới hạn nhận định	Bước kiểm tra gợi ý
Dữ liệu thiếu / Quá hạn quan trắc	- Mẫu thiếu hoặc cũ theo chính sách độ mới riêng; không suy từ heartbeat. - Số mẫu đo < 3 mẫu. - Sự cố trên bảng: Đang `OPEN`.	Dữ liệu đo đạc bị gián đoạn hoặc không đủ số mẫu liên tiếp. Hệ thống dứt khoát bảo lưu kết luận chẩn đoán tải hiện tại.	1. (*Không xâm lấn*) Giữ nguyên phiếu sự cố, không tự ý đóng phiếu. 2. (*Hạ tầng*) Kiểm tra đường truyền mạng và tiến trình dịch vụ Agent. 3. (*Theo dõi*) Chờ Agent kết nối lại và gửi mẻ đo mới để tái đánh giá.

3.4.1 Thuật toán xếp hạng bước kiểm tra (Ranked Inspection Steps Algorithm)

Danh mục các bước kiểm tra `RecommendedChecks` phải tuân thủ nghiêm ngặt 3 mức độ ưu tiên:

1. **Mức ưu tiên 1 (Không xâm lấn — Non-Invasive):** Thu thập thêm thông tin quan sát (ping IP, đối chiếu tác vụ thực hành, kiểm tra mốc thời gian) mà không gây ảnh hưởng hay gián đoạn sinh viên đang sử dụng máy.
2. **Mức ưu tiên 2 (Tác động thấp — Low-Impact):** Quan sát Task Manager trực tiếp tại chỗ, kiểm tra đèn LED cáp mạng LAN, kiểm tra nguồn điện thiết bị.
3. **Mức ưu tiên 3 (Can thiệp kỹ thuật — Invasive/Actionable):** Yêu cầu dừng tiến trình, khởi động lại dịch vụ Agent hoặc khởi động lại máy tính. **Điều kiện bắt buộc:** Phải nêu rõ lý do kỹ thuật, yêu cầu đúng thẩm quyền quản trị (`Operator` hoặc `Admin`) và phải có sự xác nhận/phê duyệt của người quản trị trước khi thực hiện.

3.5 Thiết kế giao diện thư viện tham khảo và Tiêu chí nghiệm thu (LG-D-009)

Nhằm tôn trọng quyền tự chủ kỹ thuật của Thành viên D, sổ tay không viết mã cài đặt sẵn thay cho bạn. Dưới đây là định nghĩa giao diện tham khảo và tiêu chí nghiệm thu để bạn tự tay thiết kế và lập trình:

```
namespace LabGuard.Troubleshooting;

public enum DataQualityStatus
{
    Fresh,          // Bang chung du, nguon va do moi da xac minh
    Stale,          // Bang chung cu theo chinh sach do moi rieng
    Insufficient,   // Khong du 3 mau lien tiep de ket luan
    Missing         // Thieu nguon/dinh danh bat buoc cho cau hoi
}

public sealed record RecommendedCheck(
    int StepNumber,
    string Title,
    string Description,
    string Priority,    // "LOW", "MEDIUM", "HIGH", "URGENT"
    bool IsNonInvasive
);

public sealed record TroubleshootingAssessment(
    DataQualityStatus DataQuality,
    string EvidenceSummary,
    string AssessmentConclusion,
    IReadOnlyList<RecommendedCheck> RecommendedChecks,
    IReadOnlyList<string> PermittedActions
```

```
);

public interface ITroubleshootingEngine
{
    // Ngoc tu do thiet ke kien truc noi bo (Strategy, Chain of Responsibility...)
    TroubleshootingAssessment Evaluate(TroubleshootingContext context);
}
```

Listing 4: Giao diện hợp đồng tham khảo của thư viện LabGuard.Troubleshooting

TIÊU CHÍ NGHIỆM THU (ACCEPTANCE CRITERIA)

TIÊU CHÍ NGHIỆM THU CHO THÀNH VIÊN D (ACCEPTANCE CRITERIA — LG-D-009):

- Thư viện độc lập hoàn toàn, không phụ thuộc vào ‘System.Windows.Forms‘ hay thư viện SQL client nào.
- Khi mẫu cũ hoặc chưa đủ ba mẫu liên tiếp có nguồn xác minh, trả về chất lượng phù hợp và nêu phần thiếu; không tự thay trạng thái kết nối do Central cung cấp.
- Tuân thủ chính xác ngưỡng kích hoạt và hồi phục: CPU $\geq 90\%$ (bật), $\leq 75\%$ (phục hồi); RAM $\geq 85\%$ (bật), $\leq 70\%$ (phục hồi) theo đúng chuỗi 3 mẫu liên tiếp.
- Danh sách bước kiểm tra phải sắp xếp theo thứ tự ưu tiên (bước không xâm lấn luôn đứng trước).
- Toàn bộ các kiểm thử đơn vị tự động cho 6 kịch bản sự cố (Ca 1 đến Ca 6) đều phải vượt qua (PASS 100%).

3.6 Bộ sáu ca kiểm thử dự kiến (LG-D-010)

Các ca dưới đây là **fixture giả định**, không phải API lịch sử hoặc kết quả nghiệm thu. Ca có ba mẫu chỉ được chạy tích hợp với nhận định chuỗi mẫu khi nguồn RecentMetrics đã được xác minh theo Mục 3.1.2; ở giai đoạn P0 thiếu nguồn, kết quả phải hiện thiếu bằng chứng. Mốc 20 giây là quyết định liveness của Central, không do thư viện suy từ clock client.

Table 5: Sáu ca dự kiến: dữ kiện, chất lượng và giới hạn nhận định

Ca	Dữ kiện giả định và DataQuality	Nhận định và bước kiểm tra
1	MAY-03: CPU 91.2% → 93.1% → 92.4%, RAM 58.5%. Ba mẫu đúng thứ tự/nguồn và mới trong fixture; Central báo ONLINE, HIGH_CPU/ACTIVE/OPEN. **Fresh**.	Đủ minh họa điều kiện CPU cao; không biết tiến trình gây tải. Hỏi tác vụ học; quan sát Task Manager tại máy được phép. Can thiệp chỉ khi người có quyền xác nhận.
2	MAY-07: CPU 96.5% → 62.1% → 58.4%, RAM 45.0%; ba mẫu đã xác minh; ONLINE; không có sự cố mở. **Fresh**; IncidentRecord rỗng trong ca này không tự thành Missing.	Chỉ một mẫu cao, chưa thỏa chuỗi ba mẫu mở HIGH_CPU. Theo dõi thêm, không tạo sự cố hoặc can thiệp tự động.
3	MAY-12: RAM 88.4% → 89.1% → 91.0%, CPU 24.5%; nguồn ba mẫu đã xác minh; ONLINE, HIGH_MEMORY/ACTIVE/OPEN. **Fresh**.	Nhận định tải RAM cao, không kết luận rò rỉ RAM. Đối chiếu thời gian hoạt động, kiểm tra Task Manager tại chỗ; đóng ứng dụng phải có quyền và xác nhận.

Ca	Dữ kiện giả định và DataQuality	Nhận định và bước kiểm tra (tiếp)
4	MAY-15: Central báo OFFLINE do quá hạn heartbeat hợp lệ; mẫu CPU 14.2% trước đó đã cũ theo chính sách fixture. **Stale** cho nhận định tải hiện tại. UNKNOWN là điều kiện theo ADR, cần ánh xạ B/C xác minh.	Chỉ xác nhận Central mất liên lạc; không nói đứt cáp hoặc mất điện. Người trực có thể ping bằng công cụ được phép, kiểm tra đèn cổng mạng và nguồn tại chỗ; không tự mở chức năng điều khiển từ thư viện.
5	MAY-09: mẫu cuối CPU $\geq$ 90%, RAM $\geq$ 85% đã cũ; sau đó Central báo OFFLINE. **Stale**. Chuyển ONLINE sang OFFLINE là diễn biến kết nối, không phải enum DataQuality.	Thuật lại tải cao trước khi mất liên lạc, không khẳng định quan hệ nhân quả. Kiểm tra màn hình và Agent tại chỗ; chỉ khởi động lại sau xác nhận của người có quyền và bảo toàn công việc cần thiết.
6	MAY-02: sự cố HIGH_CPU/OPEN; chỉ một mẫu 92.0% và mẫu đã cũ trong fixture. **Stale** theo ưu tiên đề xuất; EvidenceSummary vẫn phải ghi thiếu ba mẫu. Nếu thiếu nguồn bắt buộc hoàn toàn thì Missing.	Không kết luận tải hiện tại hoặc tự đóng phiếu. Hiện rõ mẫu cũ, số mẫu thiếu; kiểm tra luồng dữ liệu/Agent, chờ bằng chứng mới. Với một mẫu còn mới, ca biến thể phải trả Insufficient.

4. Năm bẫy lỗi lập trình cần tránh dành cho Thành viên D

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 1: Đồng nhất một lần đọc TCP (`NetworkStream.ReadAsync`) với một thông điệp hoàn chỉnh
Bản chất: TCP là giao thức dòng byte không ranh giới. Một bản tin JSON 500 byte có thể bị chia thành hai lần nhận (200 byte và 300 byte), hoặc ba bản tin nhịp tim liên tiếp bị gom lại trong một bộ đệm đọc.
Cách phòng tránh: Tuyệt đối không parse trực tiếp JSON từ một lệnh `ReadAsync`. Bắt buộc phải đọc đủ 4 byte Big-Endian tiền tố bằng `FrameCodec.ReadExactlyBytesAsync`, lấy đúng kích thước N, sau đó đọc chính xác tiếp N byte payload rồi mới tiến hành giải mã JSON.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 2: Phán đoán nguyên nhân võ đoán hoặc bịa đặt điểm tin cậy ảo khi dữ liệu thiếu/cũ
Bản chất: Khi thấy sự cố CPU mà nhịp tim đã mất quá hạn, lập trình viên vội vã in ra thông báo suy đoán máy trạm đang bị quá tải hoặc bị rò rỉ bộ nhớ. Đây là lỗi logic học thuật nghiêm trọng vì dữ liệu đã mất tính thời sự.
Cách phòng tránh: Tuân thủ nghiêm ngặt rào chắn kiểm tra chất lượng dữ liệu (*Data Quality Gate*). Nếu thiếu nguồn bắt buộc, mẫu cũ theo chính sách metric riêng hoặc chưa đủ ba mẫu đã xác minh, dùng quy tắc ưu tiên ở Mục 3.2 (Missing, Stale, Insufficient); không kết luận tải hiện tại. OFFLINE do Central cung cấp được hiển thị riêng, không tự đổi thành Stale chỉ vì heartbeat quá hạn.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 3: Nhầm lẫn giữa điều kiện phục hồi (`RECOVERED`) với việc đóng sự cố (`RESOLVED`)
Bản chất: Khi nhận được 3 mẫu CPU $\leq 75\%$, lập trình viên tự động cập nhật trạng thái sự cố trong CSDL thành `RESOLVED`. Điều này phá hủy quy trình kiểm soát của quản trị viên và làm mất vết lịch sử sự cố chưa được kiểm tra thực địa.
Cách phòng tránh: Phân biệt rạch ròi: `RECOVERED` là trạng thái của Tầng 2 (Điều kiện cảnh báo kỹ thuật), trong khi `RESOLVED` thuộc Tầng 3 (Quy trình vận hành). Điều kiện phục hồi chỉ mang ý nghĩa thông báo cho quản trị viên biết tải đã hạ nhiệt; chỉ có con người mới có quyền xác nhận đóng sự cố.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 4: Thư viện phân tích vi phạm ranh giới, tự ý kết nối SQL hoặc nhúng mã WinForms
Bản chất: Để tiện lấy dữ liệu, thư viện của D tự mở kết nối `SqlConnection` truy vấn bảng `Incidents`, hoặc trực tiếp thay đổi màu sắc của các Control trên WinForms. Điều này biến mã nguồn thành một mớ hỗn độn (Spaghetti Code), gây xung đột đa luồng và không thể viết Unit Test độc lập.
Cách phòng tránh: Đóng gói thư viện phân tích thành Pure C# Class Library. Toàn bộ dữ liệu đầu vào được truyền qua đối tượng ngữ cảnh (`TroubleshootingContext`), kết quả trả về là DTO thuần túy. Việc đọc DB do Thành viên B đảm trách, việc hiển thị WinForms do Thành viên C đảm trách.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 5: Rò rỉ tài nguyên Socket và nghẽn bộ nhớ do sử dụng hàng đợi không giới hạn
Bản chất: Trong các công cụ TCP Relay hoặc kiểm thử, sử dụng `Queue<T>` không giới hạn làm bộ nhớ phình to khi đối tác xử lý chậm. Đồng thời, không đóng socket đúng cách khi client ngắt kết nối khiến cổng socket bị treo ở trạng thái `CLOSE_WAIT`.
Cách phòng tránh: Luôn sử dụng `Channel.CreateBounded` với cơ chế `BoundedChannelFullMode.Wait` để áp dụng áp lực ngược. Giải phóng `TcpClient` bằng `using`; giải phóng stream đúng vòng đời, bắt kịp thời các ngoại lệ `IOException` và hủy bỏ `CancellationToken` một cách dứt khoát.

5. Bộ câu hỏi vấn đáp chuyên sâu dành cho Thành viên D (20 câu)

TÓM TẮT 30 GIÂY

Bộ câu hỏi được phân bổ cân bằng phản ánh chính xác phân công thực tế: 8 câu về nền tảng mạng TCP và kiểm thử dòng byte; 10 câu về định hướng thiết kế hỗ trợ phân tích sự cố và ranh giới kiến trúc tam giác; 2 câu về các nghiên cứu mở rộng P1 tùy chọn.

1. **Tại sao giao thức TCP lại đòi hỏi phải có cơ chế đóng khung (Framing) như tiền tố 4 byte Big-Endian?**
   Trả lời: Vì TCP là giao thức hướng dòng byte (*Byte Stream*), tầng truyền vận không bảo lưu ranh giới giữa các lần gửi tin. Tiền tố 4 byte Big-Endian chỉ rõ độ dài chính xác của gói tin

tiếp theo, giúp bên nhận gom đủ các mẩu byte bị phân mảnh trước khi tiến hành giải mã JSON, ngăn ngừa lỗi vỡ khung dữ liệu.

2. **Cơ chế Big-Endian hoạt động ra sao và hiển thị thế nào trên Wireshark?**
   Trả lời: Big-Endian lưu trữ byte có trọng số lớn nhất ở địa chỉ bộ nhớ thấp nhất (thứ tự mạng chuẩn). Ví dụ tự dựng tại Mục 2.2 có payload 197 byte; tiền tố là `00 00 00 C5`, toàn khung 201 byte. Đây không phải capture thực tế; offset tính trong khung đã tái ghép, không phải gói Ethernet.

3. **Chương trình TCP Echo (`LG-D-002`) giải quyết bài toán kỹ thuật gì trong đồ án?**
   Trả lời: TCP Echo kiểm chứng khả năng thiết lập socket lắng nghe, tiếp nhận kết nối đa luồng, đọc ghi qua `NetworkStream` và xử lý giải phóng tài nguyên an toàn khi client ngắt kết nối đột ngột, làm tiền đề tin cậy cho các phân hệ mạng phức tạp hơn.

4. **Tại sao hàng đợi trong TCP Relay (`LG-D-003`) bắt buộc phải là Bounded Channel?**
   Trả lời: Để ngăn chặn nguy cơ cạn kiệt bộ nhớ RAM khi tốc độ gửi của client lớn hơn tốc độ nhận của server đích. Hàng đợi giới hạn 100 phần tử kích hoạt cơ chế áp lực ngược (*Back-pressure*) với chính sách `FullMode.Wait`, buộc client phải tạm dừng gửi khi hàng đợi bị đầy.

5. **Làm thế nào để bắt chính xác sự kiện đối tác ngắt kết nối TCP đột ngột trong C#?**
   Trả lời: Khi đối tác đóng kết nối an toàn, lệnh `ReadAsync` trên `NetworkStream` sẽ trả về giá trị 0 (End-Of-Stream). Nếu đối tác bị rút cáp hoặc sập nguồn, socket sẽ ném ngoại lệ `IOException` hoặc `SocketException`. Bắt hai trường hợp này cho phép đóng kết nối an toàn.

6. **Bộ lọc Wireshark `tcp.port == 5000` bắt được những giai đoạn nào của phiên truyền thông?**
   Trả lời: Bắt được toàn bộ vòng đời phiên: bắt tay 3 bước (`SYN`, `SYN-ACK`, `ACK`), các khung tin dữ liệu kèm cờ `PSH, ACK`, bản tin nhịp tim định kỳ, và quá trình đóng kết nối an toàn (`FIN, ACK`) hoặc ngắt cưỡng bức (`RST`).

7. **Sự khác biệt giữa việc kiểm thử bằng stub cục bộ (`mock_harness`) và kiểm thử tích hợp mạng thật là gì?**
   Trả lời: Stub cục bộ kiểm chứng tính toàn vẹn cú pháp của hợp đồng dữ liệu trong môi trường đơn máy không trễ; kiểm thử mạng thật trên cổng 5000 đối mặt với độ trễ LAN, hiện tượng phân mảnh byte, tranh chấp kết nối đồng thời và rủi ro đứt cáp vật lý.

8. **Tại sao chu kỳ nhịp tim là 5 giây nhưng thời gian xác định mất kết nối lại là > 20 giây?**
   Trả lời: Để thiết lập khoảng dung sai an toàn chống báo động giả (*False Positive*) khi mạng phòng máy bị suy giảm chất lượng hoặc trễ tức thời. Ngưỡng 20 giây tương đương việc bỏ lỡ liên tiếp 4 bản tin nhịp tim theo đúng ADR CP2-005.

9. **Vai trò chính thức của Thành viên D trong chức năng hỗ trợ phân tích sự cố là gì?**
   Trả lời: Thiết kế và lập trình thư viện C# thuần túy (`LabGuard.Troubleshooting`) để thẩm định chất lượng bằng chứng, đối soát ma trận quy tắc giải thích tất định và xếp hạng các bước kiểm tra hỗ trợ quản trị viên trong Admin WinForms.

10. **Tại sao hệ thống hỗ trợ xử lý sự cố tuyệt đối không dùng Machine Learning hay Chatbot AI?**
    Trả lời: Vì trong giám sát phòng máy, quản trị viên yêu cầu tính giải thích minh bạch, khả năng đối chiếu bằng chứng và kiểm tra quy tắc. Mô hình học máy dạng hộp đen hoặc chatbot có thể tạo ra ảo giác (hallucination), suy đoán sai lệch và không thể kiểm chứng bằng unit test tự động.

11. **Thư viện của D lấy dữ liệu từ đâu và có được phép truy vấn trực tiếp SQL Server không?**
    Trả lời: Tuyệt đối không được truy vấn SQL Server trực tiếp. Thư viện chỉ nhận dữ liệu có

thẩm quyền từ Central Server được chuyển tiếp qua Admin thông qua đối tượng ngữ cảnh `TroubleshootingContext`.

12. **Thế nào là dữ liệu STALE và thư viện xử lý tình huống này ra sao?**
Trả lời: Stale mô tả bằng chứng đã cũ theo chính sách độ mới metric cần được nhóm chốt. Không dùng nó thay cho OFFLINE: Central quyết định liveness bằng đồng hồ đơn điệu. Thư viện nêu dữ kiện cũ và giới hạn nhận định, không tự kết luận nguyên nhân hoặc đổi trạng thái kết nối.

13. **Tại sao thuật toán yêu cầu tối thiểu 3 mẫu metric liên tiếp mới đưa ra đánh giá tải tài nguyên?**
Trả lời: Một hoặc hai mẫu đo cao có thể chỉ là hiện tượng đột biến nhất thời (*Spike*) khi mở ứng dụng. Cần tối thiểu 3 mẫu có nguồn và tính liên tiếp đã được xác minh (khoảng đầu–cuối 10 giây nếu cách đều 5 giây) để minh họa tải cao kéo dài; P0 không tự cung cấp lịch sử này (*Sustained Overload*), tránh đưa ra các khuyến nghị kiểm tra không cần thiết.

14. **Hãy phân biệt rõ 3 tầng trạng thái: Thiết bị, Điều kiện kỹ thuật và Quy trình sự cố?**
Trả lời: Thiết bị là trạng thái tầng mạng (ONLINE/OFFLINE qua socket/nhịp tim). Điều kiện kỹ thuật là trạng thái ngưỡng tài nguyên (ACTIVE/RECOVERED/UNKNOWN theo Hysteresis của RuleEngine). Quy trình sự cố là trạng thái nghiệp vụ trong CSDL (OPEN/ACKNOWLEDGED/INVESTIGATING/RESOLVED do con người thao tác).

15. **Khi điều kiện kỹ thuật chuyển sang RECOVERED, sự cố trên Admin có tự đóng thành RESOLVED không?**
Trả lời: Tuyệt đối không. Điều kiện `RECOVERED` chỉ báo hiệu tài nguyên đã hạ nhiệt an toàn. Sự cố trên Admin vẫn giữ nguyên ở quy trình (`OPEN` hoặc `INVESTIGATING`) để người quản trị kiểm tra nguyên nhân gốc rễ và chủ động gửi lệnh `INCIDENT_RESOLVE`.

16. **Nguyên tắc xếp hạng các bước kiểm tra (`RecommendedChecks`) được xây dựng theo tiêu chí nào?**
Trả lời: Ưu tiên bước không xâm lấn (*Non-invasive*) lên đầu tiên (hỏi người dùng, xem Task Manager, kiểm tra ping); kế tiếp là bước tác động thấp (kiểm tra service, cáp mạng); cuối cùng mới đến can thiệp có điều kiện (yêu cầu dừng tiến trình, khởi động lại máy khi có xác nhận của quản trị viên).

17. **Tại sao thư viện không được phép đưa ra điểm số phần trăm tin cậy võ đoán?**
Trả lời: Vì không có mô hình xác suất thực nghiệm được kiểm chứng khoa học phía sau để bảo đảm tính chuẩn xác của các con số phần trăm. Hệ thống dùng kết luận logic nhị phân rõ ràng gắn liền với bằng chứng định lượng cụ thể.

18. **Sáu kịch bản kiểm thử (Ca 1 đến Ca 6) đóng vai trò gì trong việc tích hợp nhóm?**
Trả lời: Đóng vai trò bộ kiểm thử biên chuẩn hóa để đối soát hợp đồng giữa thư viện của D, dữ liệu Central của Quang (B) và giao diện Admin của Vy (C), kiểm tra những tình huống đã nêu khi có đủ nguồn dữ liệu; không chứng minh mọi tình huống thực tế hoặc tích hợp đã hoàn tất.

19. **Sự khác biệt kỹ thuật giữa L4 TCP Gateway và L7 Collector trong nghiên cứu mở rộng P1 là gì?**
Trả lời: L4 Gateway chuyển tiếp luồng byte thuần túy ở tầng giao vận mà không đọc nội dung; L7 Collector giải mã từng khung tin JSON ở tầng ứng dụng, gom cụm nhiều số liệu nhỏ thành khối lớn để tối ưu hóa lưu lượng mạng WAN.

20. **Bộ giả lập Simulator 50 máy ảo trong Phụ lục P1 giúp ích gì cho đồ án?**
Trả lời: Giúp nhóm có thể mô phỏng tải của hàng chục máy trạm đồng thời trên một máy phát

triển duy nhất thông qua cơ chế bất đồng bộ `Task.Run`, hỗ trợ kiểm tra năng lực chịu tải của máy chủ mà không cần 50 máy tính thật.

Phụ lục: Nghiên cứu mở rộng P1 tùy chọn (Gateway, Collector & Simulator)

TÓM TẮT 30 GIÂY

Phần phụ lục này lưu trữ các kiến thức nghiên cứu mở rộng về mặt kỹ thuật mạng quy mô lớn. Các nội dung này là **nghiên cứu tùy chọn cấp độ P1**, không nằm trong cam kết bắt buộc của Thành viên D để hoàn thành đồ án cốt lõi.

Phụ lục A: Cân bằng tải tầng 4 (Gateway L4) và Gom cụm thông điệp (Collector L7)

Khi quy mô phòng máy mở rộng lên hàng nghìn máy trạm phân bố qua nhiều lớp mạng router/NAT:

- **Gateway tầng giao vận (L4 TCP Load Balancer):** Hoạt động như một bộ chuyển tiếp kết nối thô. Gateway tiếp nhận kết nối TCP từ Agent và phân phối tới các nút Central Server theo giải thuật *Least-Connections*. Điểm mạnh là thông lượng cực cao do không phải giải mã nội dung gói tin; điểm yếu là không thể định tuyến dựa trên nội dung loại thông điệp.
- **Collector tầng ứng dụng (L7 Message Multiplexer):** Đóng vai trò trạm gom dữ liệu trung gian trong từng phòng học. Collector tiếp nhận hàng chục kết nối Agent, giải mã thông điệp qua `FrameCodec`, đưa vào hàng đệm `BoundedChannel`, sau đó gom thành từng gói dữ liệu đo đạc lớn (*Batch Multiplexing*) chuyển tiếp về Central qua một kết nối WAN duy nhất. Cơ chế này giảm đáng kể chi phí mở kết nối đồng thời trên máy chủ trung tâm.

Phụ lục B: Kỹ thuật giả lập tải phòng máy quy mô lớn (LabGuard.Simulator)

Dự án `LabGuard.Simulator` là công cụ hỗ trợ kiểm thử tải dự kiến triển khai sau khi MVP Core hoàn tất:

- **Mô hình VirtualAgent nhẹ nhàng:** Thay vì khởi tạo 50 Thread hệ điều hành gây lãng phí bộ nhớ và nghẽn CPU do chuyển ngữ cảnh (*Context Switching*), Simulator tận dụng cơ chế I/O bất đồng bộ (`async/await`) kết hợp với `PeriodicTimer(5s)` để vận hành 50 máy ảo nhẹ trên chỉ vài luồng của hệ thống.
- **Cơ chế bơm lỗi chủ động (Fault Injection):** Tích hợp bàn phím điều khiển nhằm chủ động tạo các tình huống bất thường:
  - Bơm 3 mẫu CPU 95% cho một máy ảo cụ thể để kiểm tra thuật toán phát hiện sự cố của Central.
  - Bơm chuỗi CPU hạ nhiệt $\leq 75\%$ để kiểm chứng điều kiện phục hồi Hysteresis.
  - Ngắt kết nối socket đột ngột của một máy ảo để kiểm tra cơ chế cảnh báo mất nhịp tim > 20s trên Admin Dashboard.

— HẾT CẨM NANG HƯỚNG DẪN THÀNH VIÊN D —